



ACCELERATE 2026

INSPIRATION GUIDE

Powered by  hackerearth





Theme 1

ABB Accelerate 2026

Theme 1: Agentic Predictive Maintenance Studio

Sample Idea for Theme 1

Idea Name: **FocusPane**

An Adaptive, Context-Aware Control Interface for Industrial Operators

1. Team Details

TEAM

Team Name: [Team Name]

Team Members:

- [Name of first team member]
- [Name of second team member (if any)]
- [Name of third team member (if any)]
- [Name of fourth team member (if any)]
- [Name of fifth team member (if any)]

2. Problem Statement & Relevance

CONTEXT

Selected Theme: Theme 1 - Next-Generation Control System Interface

Industrial control rooms today are built around exposing everything the underlying system produces - every tag, every alarm, every trend, and leaving the operator to work out what matters. During a normal shift this is manageable; during an upset, when dozens of correlated alarms fire within seconds, it becomes genuinely dangerous. Operators lose time hunting for the signal in the noise, and the interface itself becomes a source of cognitive load instead of a decision-support tool. FocusPane is built to close exactly this gap: it re-orders, groups, and explains what the control system already knows, so the operator sees the three things that matter right now instead of the three hundred things the system is capable of showing.

FocusPane does not replace the DCS/SCADA or its alarm database, it sits as an intelligence and presentation layer on top of the existing historian and alarm feed, so it can be piloted on a single unit or process area without touching certified control logic. This keeps adoption risk low for an operator-facing tool in a safety-critical environment.

Why this problem

- Alarm floods during process upsets routinely exceed the rate an operator can meaningfully process, increasing the risk of a missed critical alarm.
- Interfaces are typically designed around tag structure and system architecture, not around the operator's actual decision sequence.
- The same screen is shown to a junior operator, a senior operator, and a shift supervisor, even though each needs a different level of detail and a different set of controls.
- Existing HMI upgrades tend to focus on visual polish rather than on reducing the operator's time-to-decision, which is the metric that actually matters during an incident.

3. Idea and Approach

SOLUTION

Solution Overview

FocusPane ingests live alarm and process data alongside the plant's alarm rationalization documentation, correlates related alarms into a single 'event,' and ranks events by process and safety impact rather than by raw timestamp. The operator's screen is reorganized in real time around these ranked events, with the interface adapting layout, terminology, and permitted actions to the logged-in role. Every recommendation the system surfaces is accompanied by the underlying data and a plain-language reason, so operators can verify the system's reasoning rather than being asked to trust it blindly.

Key Features

- Continuously groups related alarms into a single correlated event using tag topology and time-proximity, cutting the number of items an operator must individually triage.
- **Impact-Ranked Event Queue:** Ranks active events by estimated process and safety impact, not arrival order.
- **Role-Adaptive Layout:** Screen layout, alarm density, and available controls change automatically based on operator role (trainee, operator, shift supervisor).
- **Context Panel:** A concise, human-readable summary of what changed, what it likely means, and what the recommended next check is.
- **Suggested-Action Rail:** Recommended next actions are drawn from historical resolution patterns for similar events and always shown with the option to dismiss and act independently.
- **Situation Replay:** A rewind-and-replay view of the last N minutes of an event, so shift handover and post-incident review don't require digging through raw historian trends.

4. Technical Solution & Innovation

TECHNICAL

Architecture Overview

- **Data Ingestion Layer:** Reads live tags, alarms, and events from the DCS/SCADA historian via OPC UA, without writing back to control logic.
- **Correlation Engine:** Groups temporally and topologically related alarms into single correlated events using a graph built from the plant's alarm rationalization data.
- **Impact Scoring Layer:** A lightweight scoring model that ranks correlated events by estimated safety/process impact using rationalization severity, historical outcome data, and current process state.
- **Reasoning & Explanation Layer:** Turns the top-ranked events and their scores into a plain-language summary and recommended checks.
- **Presentation Layer:** Renders the adaptive, role-aware interface and pushes updates to operator stations in real time.

Model

Type: Hybrid rule-based correlation + learned ranking model

- Deterministic correlation rules seeded from the plant's existing alarm rationalization matrix, so behaviour is auditable from day one.
- A gradient-boosted ranking model (LightGBM) trained on historical alarm-to-resolution sequences to prioritize events by likely impact.
- A small language model constrained to the plant's tag/alarm vocabulary to generate the plain-language context summaries, with all facts sourced from structured data rather than free generation.

Technical Stack

- **Frontend:** React, renders the adaptive operator display and role-based views.
- **Backend:** Node.js, FastAPI, orchestrates ingestion, correlation, and the reasoning pipeline.
- **AI/ML:** Python, LightGBM, scikit-learn, powers alarm correlation and impact ranking.
- **Integration:** OPC UA client libraries, integrates with the plant historian/SCADA without modifying control logic.
- **Cloud/Edge:** Azure / on-prem edge gateway, supports in-plant deployment for latency and data-residency needs.
- **Database:** TimescaleDB, PostgreSQL, stores time-series alarm/event data and role/configuration data.

Innovation Highlights

- ♦ **Impact-first triage:** Prioritizes by estimated impact rather than presenting alarms in arrival order, directly targeting operator time-to-decision instead of screen aesthetics.
- ♦ **Explainability by construction:** Explanation is generated from structured data at every step, so the interface can be audited and trusted in a safety-critical setting.
- ♦ **One data model, many roles:** The same underlying event and ranking data drives three different views (trainee, operator, supervisor), rather than three separately maintained screens.
- ♦ **Non-invasive integration:** Runs alongside the existing DCS/SCADA as a read-only overlay, making it deployable on brownfield sites without a control-system change request.

5. Implementation & Impact

IMPACT

Future Improvements

- Extend impact scoring with plant-specific safety instrumented system (SIS) data for a more precise severity model.
- Add voice-driven query support so operators can ask the context panel direct questions during high-load situations.
- Build a configuration studio so plant engineers can tune correlation rules without vendor involvement.
- Introduce a closed-loop feedback mechanism where operator overrides retrain the ranking model over time.

Expected Business Value

- Reduced operator time-to-decision during alarm floods and process upsets.
- Lower risk of missed or delayed response to safety-critical alarms.
- Faster, more consistent shift handovers through the situation replay view.
- Reduced training ramp-up time for new operators, since the interface itself surfaces context that previously lived only in senior operators' experience.

6. Methodology / Architecture Diagram

ARCHITECTURE

[Insert architecture diagram / link here]

SUBMIT HERE



Theme 2

ABB Accelerate 2026

Theme 2: Multimodal Maintenance Intelligence Agent

Sample Idea for Theme 2

Idea Name: **TopoSense**

Multi-Agent Kubernetes Discovery, Dependency Mapping & Root-Cause Reasoning

1. Team Details

TEAM

Team Name: [Team Name]

Team Members:

- [Name of first team member]
- [Name of second team member (if any)]
- [Name of third team member (if any)]
- [Name of fourth team member (if any)]
- [Name of fifth team member (if any)]

2. Problem Statement & Relevance

CONTEXT

Selected Theme: Theme 2 - Beyond Monitoring: AI Agents for Real-Time Pod Resource Discovery and Dependency Mapping

Standard Kubernetes monitoring tells an engineer that a pod is unhealthy, that CPU is high, or that a service's latency spiked, but it does not tell them how that pod relates to everything else in the cluster, or where a failure actually originated once it starts cascading through dependent services. Engineers currently reconstruct this picture manually, either from memory or from hand-maintained topology diagrams that are out of date the moment someone ships a new deployment. TopoSense is built to remove that manual step entirely: it discovers cluster resources and their relationships continuously, keeps a live dependency graph, and uses a small team of cooperating AI agents to reason over that graph when something breaks.

The system is scoped to discovery, dependency mapping, and root-cause reasoning; it does not attempt to be a full observability replacement or a general-purpose APM tool. It is designed to sit alongside existing monitoring (metrics, logs, traces) and add the relationship layer that is currently missing, so it can be adopted incrementally without displacing an organization's existing observability stack.

Why this problem

- Modern services are distributed across dozens to hundreds of pods, and dependencies shift with every deployment, making manual topology tracking unreliable.
- Existing monitoring tools report symptoms (high CPU, restarts, latency) but rarely answer 'what does this affect, and what actually caused it.'
- During an incident, engineers spend a disproportionate share of resolution time simply reconstructing what is connected to what, before root-cause work even begins.
- Manually configured topology tools go stale immediately in environments with frequent deployments, autoscaling, and ephemeral pods.

3. Idea and Approach

SOLUTION

Solution Overview

TopoSense continuously discovers pods, services, and their network/data relationships directly from the Kubernetes API and live traffic, builds a real-time dependency graph, and detects when that graph changes in ways that matter (a new dependency appears, a critical path disappears, a service starts fanning out unusually). When an anomaly or failure is detected, a coordinated set of AI agents traces the failure across the graph, identifies the most likely origin, and produces a confidence-scored, plain-English explanation rather than a raw list of alerts.

Key Features

- **Automatic Resource Discovery:** Continuously builds the pod/service dependency graph from the Kubernetes API and observed traffic, with zero manual topology configuration.
- **Real-Time Dependency Graph:** A live, queryable graph of how every pod and service in the cluster is actually connected right now.
- **Change & Anomaly Detection:** Flags meaningful shifts in the dependency graph itself (new dependency, broken critical path, unusual fan-out), not just resource-level metric spikes.
- **Multi-Agent Root-Cause Tracing:** Traces a failure backward through the dependency graph to identify the most likely origin point, with a confidence score.
- **Impact/Blast-Radius View:** Shows every service and pod that is downstream of a failing component, so engineers immediately know the blast radius.
- **Natural-Language Incident Explanation:** Converts the agents' graph-walk into a short, plain-English incident summary an on-call engineer can act on in seconds.

4. Technical Solution & Innovation

TECHNICAL

Architecture Overview

- **Discovery Layer:** Watches the Kubernetes API (pods, services, endpoints, ingress) and correlates it with sampled network flow data to detect real, not just declared, dependencies.
- **Graph Construction & Storage:** Maintains a continuously updated graph database of resources and their relationships, versioned so past topology states can be replayed.
- **Telemetry & Anomaly Layer:** Streams pod/service health signals and detects graph-level anomalies (structural changes, unusual traffic patterns) in addition to standard metric thresholds.
- **Multi-Agent Reasoning Layer:** A discovery agent, a correlation agent, and a root-cause agent cooperate over the dependency graph: the first keeps the graph current, the second links symptoms to graph nodes, and the third walks the graph backward to the likely origin.
- **Presentation & API Layer:** Surfaces the live graph, blast-radius view, and incident explanations to engineers, and exposes the same data via API for existing dashboards.

Model

Type: Graph-based dependency model + coordinated LLM agents

- Graph algorithms (shortest-path, centrality, community detection) identify critical paths and candidate root-cause nodes.
- An anomaly-detection model (isolation forest / seasonal baselining) flags unusual structural or traffic changes in the graph.
- Coordinated LLM-based agents, grounded in the graph and telemetry data, perform the root-cause walk and produce the explanation; each agent's output is checked against the underlying graph data rather than generated freely.

Technical Stack

- **Frontend:** React, powers the live dependency graph explorer and incident view.
- **Backend:** Python, FastAPI, orchestrates discovery, the agent pipeline, and the API layer.
- **Agent Orchestration:** LangGraph / custom orchestration, coordinates the discovery, correlation, and root-cause agents.
- **Graph Database:** Neo4j, stores and queries the live dependency graph and its historical versions.
- **Cluster Integration:** Kubernetes API client, eBPF-based flow sampling (Cilium/Hubble), source real and declared dependency data.
- **Telemetry:** Prometheus-compatible metrics store, ingests existing telemetry so TopoSense augments rather than replaces current monitoring.

Innovation Highlights

- ♦ **Zero manual topology configuration:** Builds the dependency graph from live traffic and API state rather than requiring engineers to declare topology, so it stays accurate through every deployment.
- ♦ **Agents reason over structure, not just metrics:** Agents don't just alert on symptoms, they walk the actual dependency graph to identify where a cascade started.
- ♦ **Confidence-scored, explainable output:** Every root-cause claim ships with a confidence score and the graph path that produced it, so engineers can verify rather than blindly trust the output.
- ♦ **Additive, not a replacement:** Designed as an additive layer over existing Prometheus/Grafana-style monitoring, lowering adoption friction for teams with an established observability stack.

5. Implementation & Impact

IMPACT

Future Improvements

- Extend discovery to cross-cluster and multi-cloud dependency mapping.
- Add predictive alerts that flag structurally risky changes (e.g., a new single point of failure) before they cause an incident.
- Introduce agent-proposed remediation actions (scale, restart, roll back) with human approval before execution.
- Build a historical topology diff view for post-incident and change-management review.

Expected Business Value

- Reduced mean-time-to-diagnosis by removing manual topology reconstruction from the incident-response path.
- Fewer repeat incidents from unrecognized single points of failure, surfaced proactively through graph analysis.
- Lower on-call cognitive load through plain-English, confidence-scored incident explanations.
- Faster onboarding for engineers new to a service, since the live dependency graph replaces tribal knowledge of 'what talks to what.'

6. Methodology / Architecture Diagram

ARCHITECTURE

[Insert architecture diagram / link here]

SUBMIT HERE